

Day 51: Cart Class

Day 51: Cart Class

1. Day Number

Day 51

2. Topic Name

Class with List Attribute

Today you will learn how an object can contain a **list as one of its attributes**.

3. Connection

Yesterday, you created a Product class containing information such as:

- product name
- price
- stock

Today, you will create a new class called Cart.

A cart needs to hold multiple products, so it makes sense for the cart to contain a **list of items**.

Conceptually:

```
Product
|
| added later
v
Cart
|
---> items = []
```

For today, you are **not adding products yet**.

You are only creating the cart with an empty list.

4. Important Topics

Class

A class is a blueprint for creating objects.

Example structure:

```
class SomeClass:
    ...
```

Constructor

The constructor initializes an object when it is created.

In Python:

```
def __init__(self):
```

For today's cart, the constructor will create the list used for storing items.

List Attribute

An attribute does not have to contain only strings or numbers.

It can also contain a list.

For example:

```
self.tasks = []
```

Here:

- tasks is an attribute
- [] means an empty list
- every object can have its own list

This same idea will be used for the cart.

5. Foundational Notes

Until now, you may have used attributes like:

```
self.name  
self.price  
self.stock
```

These normally contain single values.

But attributes can contain different types of data:

```
String  
Integer  
Float  
Boolean  
List  
Dictionary  
Other objects
```

For example:

```
self.name = "Rahul"  
self.age = 30  
self.skills = []
```

Here, `skills` is a list attribute.

For a shopping cart, we can think of:

```
Cart Object
├── items
│   └── []
```

Initially, the cart contains nothing.

Therefore:

```
items = []
```

Later, you could imagine:

```
items = [product1, product2, product3]
```

But that is **not required today**.

Important idea

The list should normally be created inside the constructor:

```
def __init__(self):
    self.some_list = []
```

This means that when a new object is created, it starts with its own empty list.

6. Easy Example

Consider a simple `Playlist` class.

A new playlist can start with an empty list of songs.

```
class Playlist:
    def __init__(self):
        self.songs = []

my_playlist = Playlist()

print(my_playlist.songs)
```

Output:

```
[]
```

Here:

```
Playlist
├── songs
```

```
|  
└─ []
```

The same idea can be applied to a shopping cart.

7. Problem Statement

Create a Cart class.

The class should contain an attribute called:

```
items
```

When a new cart is created, `items` should be an **empty list**.

Then:

1. Create one Cart object.
2. Print the cart's `items`.

The result should show an empty list.

8. Concepts Used

You will use:

- class creation
- `__init__()` constructor
- `self`
- object creation
- attributes
- lists
- empty list `[]`
- accessing an object attribute
- `print()`

Main concept:

```
Object  
└─ Attribute  
    └─ List
```

9. Thought Process

Before writing code, think about the problem step by step.

Step 1: What object do I need?

You need a shopping cart.

Therefore, you need a:

```
Cart class
```

Step 2: What information should the cart store?

A cart stores items.

So you need an attribute:

```
items
```

Step 3: What should a new cart contain?

A newly created cart should contain no products.

Therefore:

```
items = empty list
```

Conceptually:

```
items = []
```

Step 4: Where should items be created?

It should be initialized when the cart object is created.

Therefore, initialize it inside:

```
constructor
```

Step 5: Create the object

Create one object from the Cart class.

Conceptually:

```
my_cart = Cart object
```

Step 6: Print the items

Access the object's list attribute and print it.

Expected value:

```
[]
```

10. Pseudocode

```
START
```

```
Create Cart class
```

Create constructor

Create items attribute

Set items to an empty list

Create one Cart object

Print object's items attribute

END

Visual flow:

```
Create Cart class
  |
  v
Create constructor
  |
  v
items = empty list
  |
  v
Create Cart object
  |
  v
Access cart.items
  |
  v
Print []
```

11. Suggested Solving Approach

Use an **OOP-based approach**.

Your structure should roughly be:

```
Class
  |
  └─ Constructor
        |
        └─ List attribute

Create Object
  |
  └─ Access attribute
        |
        └─ Print
```

You do not need:

- functions outside the class
- loops

- conditions
- user input
- product adding logic
- removal logic

Keep today's program very small.

12. Easy Edge Cases

Empty Cart

This is actually the normal starting condition for today's program.

```
items = []
```

Printing it should show:

```
[]
```

There is no need to handle additional edge cases yet because today's cart does not perform any operations.

Later, you may handle situations such as:

```
adding products  
removing products  
empty-cart checkout  
product quantity
```

But not today.

13. Expected Input

No keyboard input is required.

The program simply creates a cart object.

Conceptually:

```
Create one Cart object
```

14. Expected Output

The cart starts empty, so the output should look like:

```
[]
```

This confirms that:

```
Cart object  
|
```

```
└─ items
    └─ empty list
```

15. Hint Only

Think about the Playlist example:

```
class Playlist:
    def __init__(self):
        self.songs = []
```

For your problem, replace the idea of:

```
Playlist → Cart
songs      → items
```

Then create one object and access the list using:

```
object_name.attribute_name
```

Your target structure is:

```
Cart
└─ __init__()
    └─ items = []
```

Do not add an `add_item()` method yet. For Day 51, your only goal is to create a Cart object whose `items` attribute starts as an empty list.